Partie 1 : Technologie React

Semaine 1
Píesentation de React
Enviíonnement de développement

Semaine 2
Développement avec JSX Cíéation des Composants

Semaine 3
Configuíation des composants : Píops, Etat Local

Semaine 4
Inteíaction avec Utilisateuí : Evenements, Ïoímulaiíes

Semaine 5
Communiqueí avec seíveuí Routage

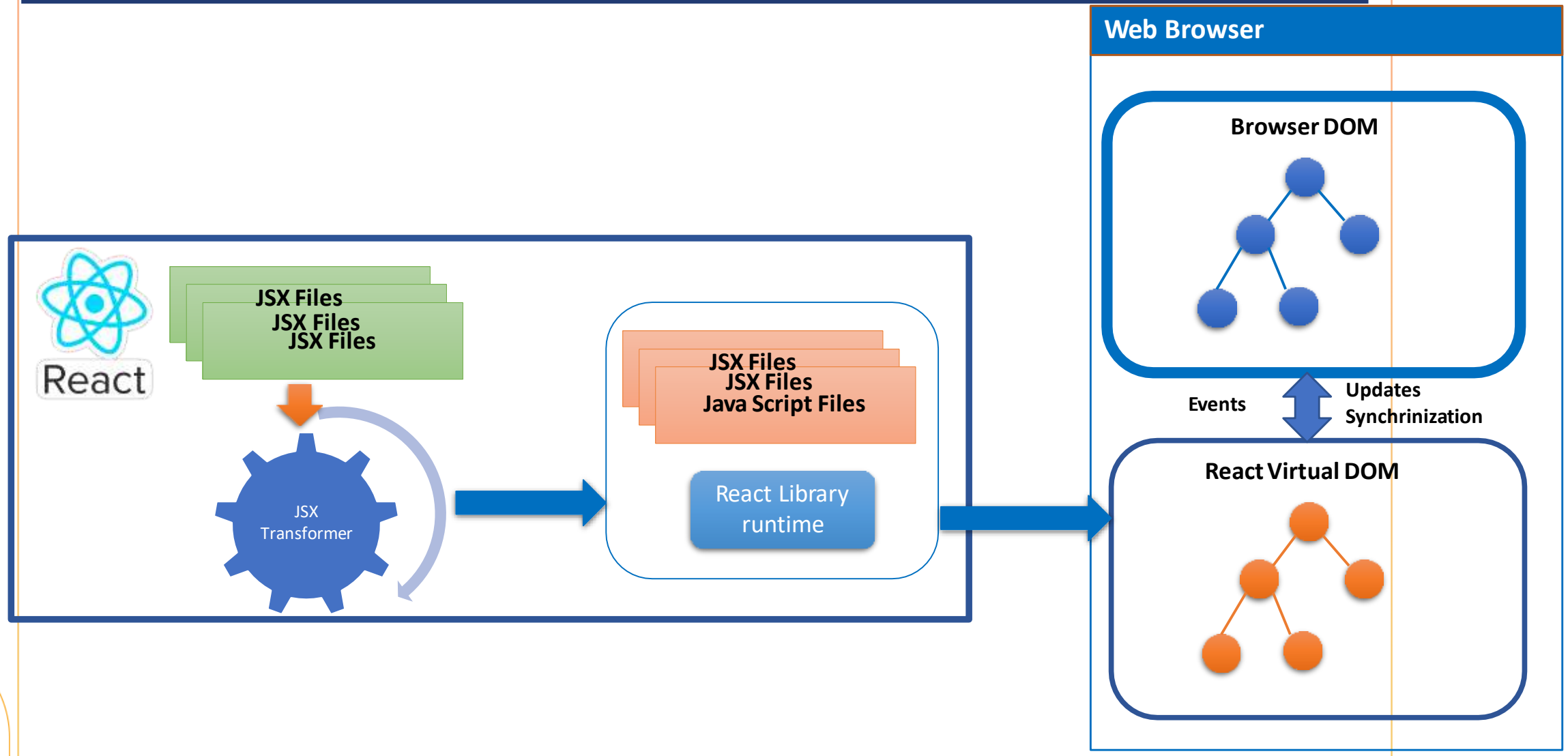
Semaine 6
Communication inteí-composants

Développement avec JSX

Création des Composants

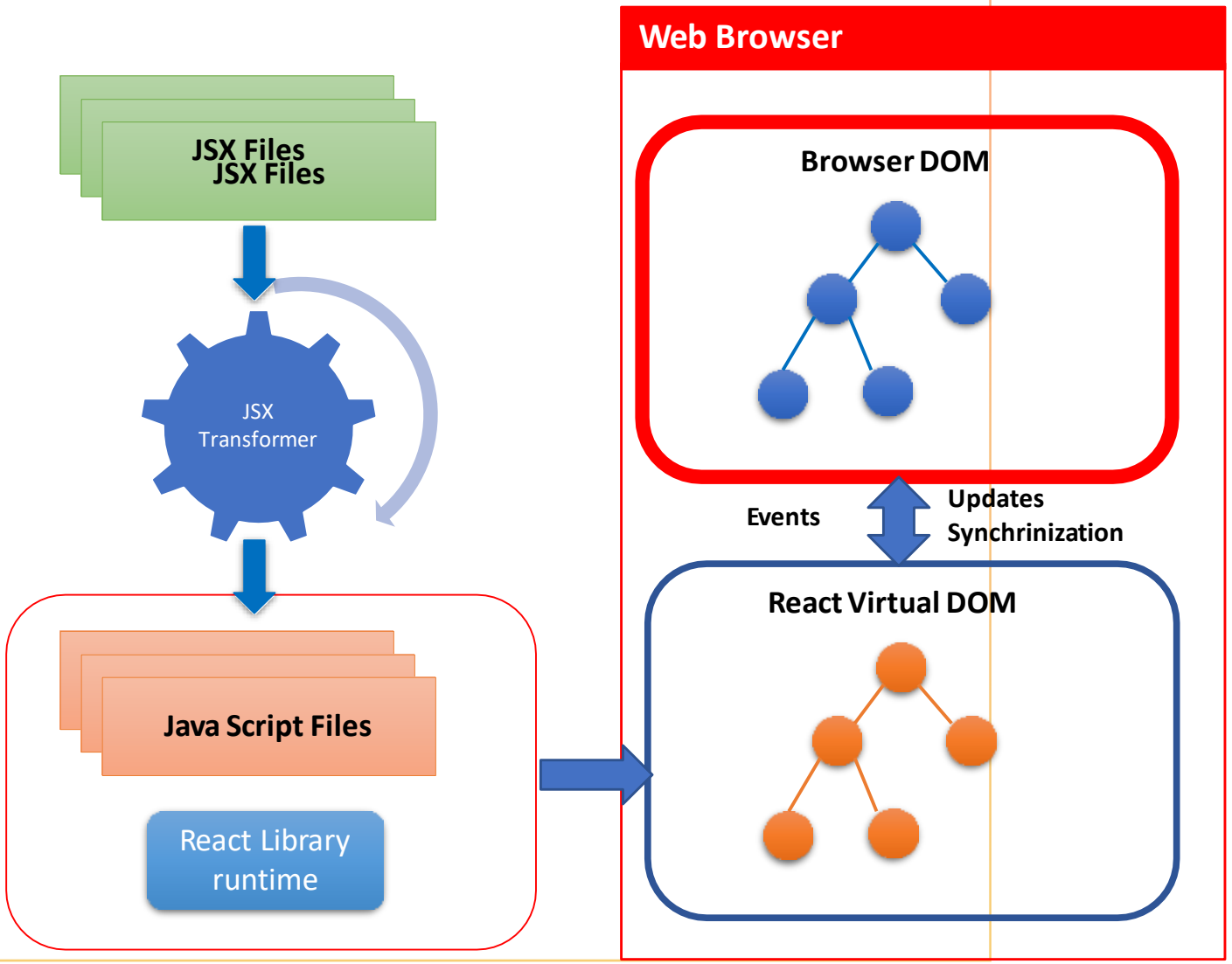
UNIVERSIAPOLIS
Université Internationale d'Agadir
الجامعة الدولية لأكادير
ECOLE POLYTECHNIQUE D'AGADIR

ReactJS


Web Browser
Browser DOM
React
JSX Files
JSX Files
JSX Files
JSX
Transformer
JSX Files
JSX Files
Java Script Files
React Library
runtime
Events
Updates
Synchrinization
React Virtual DOM

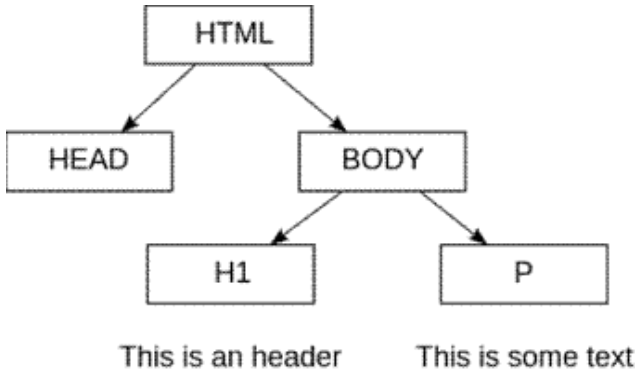
ReactJS

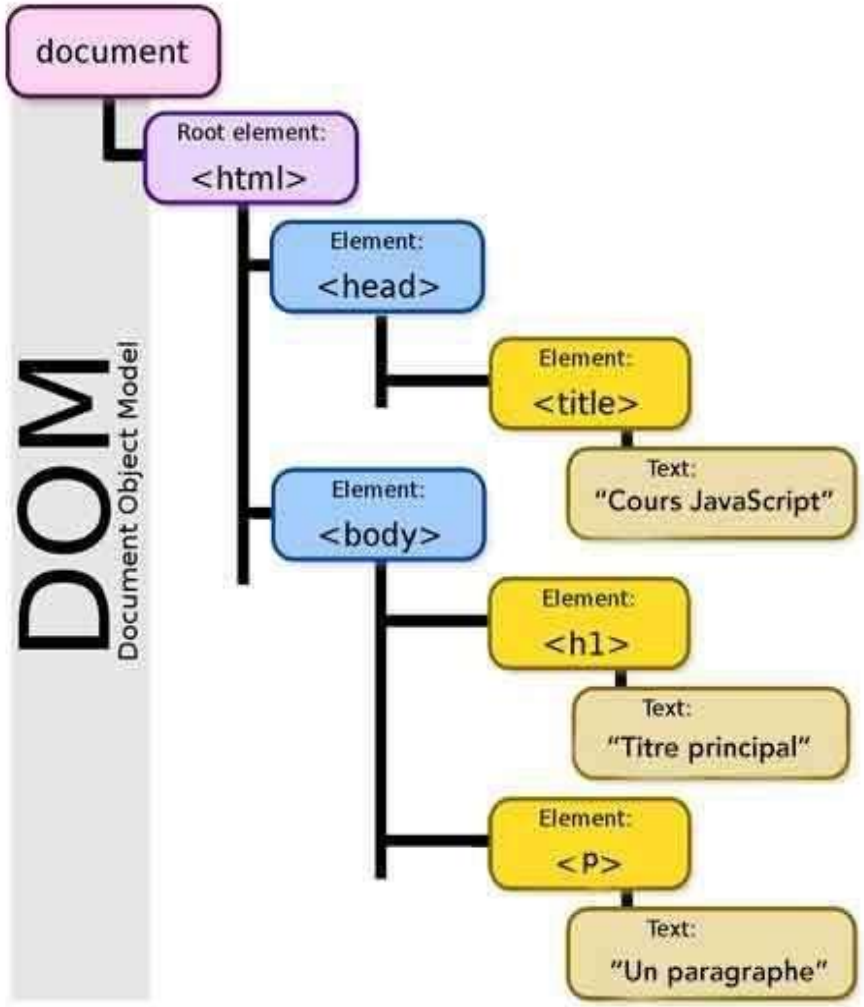
- Le moteur de rendu JSX, permet de coupler, dans un même fichier JSX, les trois aspects d'un composant React à savoir :
  - **L'état du composant** (Component State) qui représente le Modèle
  - **Le comportement** (Behaviour ou Controller)
  - **Structure de la vue du composant** (View)
- Il aussi possible de programmer les applications React en utilisant Type Script
- Le Moteur de React génère un DOM virtuel qui sera transformé et synchronisé avec le DOM réel du Navigateur
- Le Virtual DOM optimise les changements à apporter au niveau du Browser DOM. Ce qui lui permet d'accélérer l'aspect réactif du rendu



Virtual DOM

Le **DOM** (**D**ocument **O**bject **M**odel) est l'API qui permet de manipulé le contenu hiérarchisé d'une page web (c-à-d du HTML) sous forme d'arbre. Les éléments HTML constituent les nœuds de cet arbre.

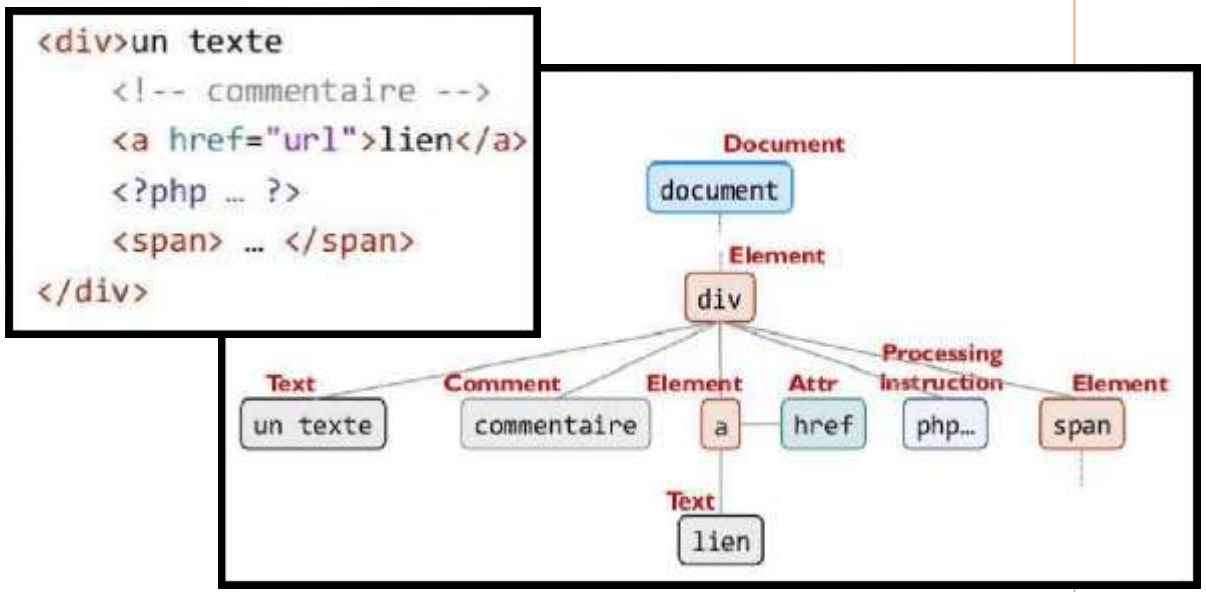




Virtual DOM

- Le DOM est une interface de programmation qui permet à des scripts d'examiner et de modifier le contenu du navigateur web.
- Un document XML ou HTML peut être vu sous la forme d'un arbre.
- Chaque nœud de l'arbre est considéré comme un objet.

- Les principaux types de nœuds sont:
  - **Document** : la racine
  - **Element** : les balises standards
  - **Text** : le texte
  - **Comment** : les commentaires
  - **Attr** : les attributs des balises
  - **Processing instructions** : les balises spécifiques comme par exemple <?php...?>



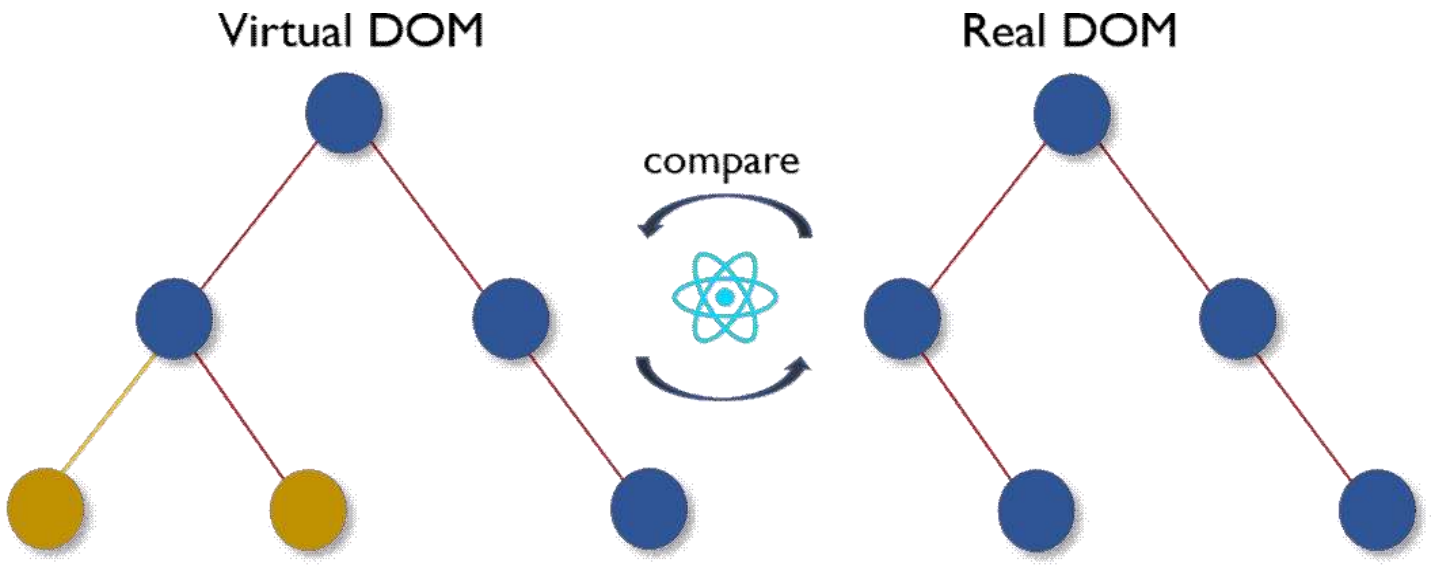
Virtual DOM

- Le DOM HTML est une norme standard qui définit :
  - Les éléments HTML comme des objets
  - Les propriétés de tous les éléments (les valeurs qu'on peut définir)
  - Les méthodes pour accéder aux éléments
  - Les événements qu'on peut effectuer sur les éléments,
- Le DOM offre des fonctions appropriées afin de trouver les éléments :

Méthode	Description
document.**getElementById**(id)	Trouve un élément par son identificateur
document.**getElementsByTagName**(nom)	Trouve des balises via leur nom
document.**getElementsByClassName**(nom)	Trouve des balises via le nom de leur class CSS

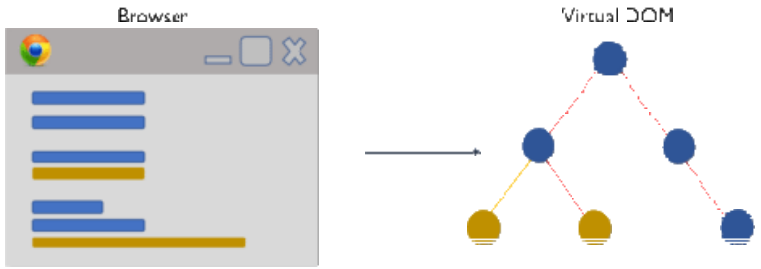
Virtual DOM

React utilise le concept du **DOM** virtuel. Le **DOM virtuel** (VDOM) est un concept de programmation dans lequel une représentation idéale, ou « virtuelle », d'une interface utilisateur (UI) est conservée en mémoire et synchronisée avec le DOM « réel » de la page.

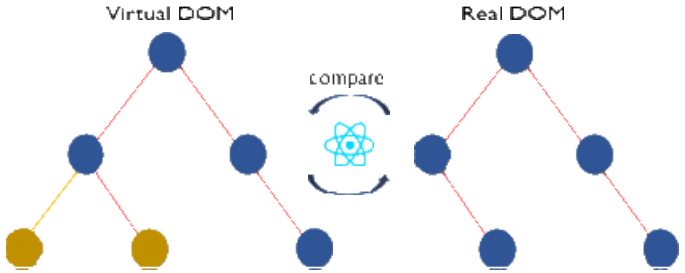


Virtual DOM

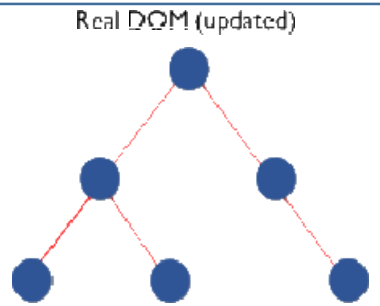
- Chaque fois que des données changent, l'interface utilisateur entière est restituée en représentation Virtual DOM



- La différence entre la représentation DOM précédente et la nouvelle est calculée

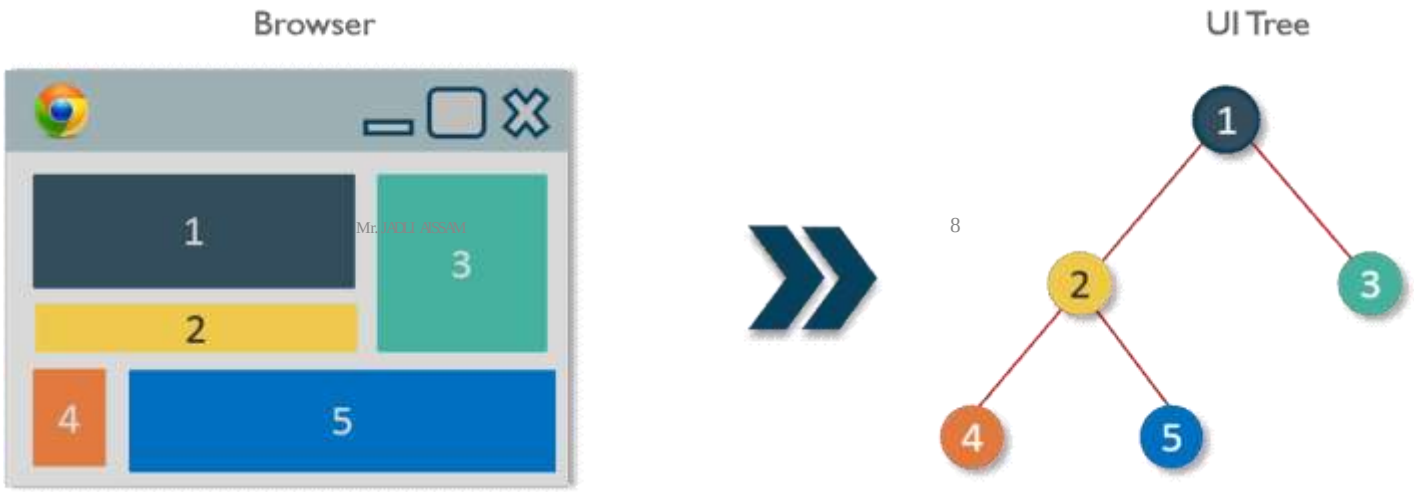


- Une fois les calculs effectués, le DOM réel va mettre à jour uniquement les objets qui ont réellement changé.



React : Composants

React est basé sur l'idée que **tout est un composant**. Il est basé sur la conclusion qu'une interface utilisateur est une combinaison de plusieurs composants, chaque composant pouvant être la combinaison d'autres petits composants, etc.



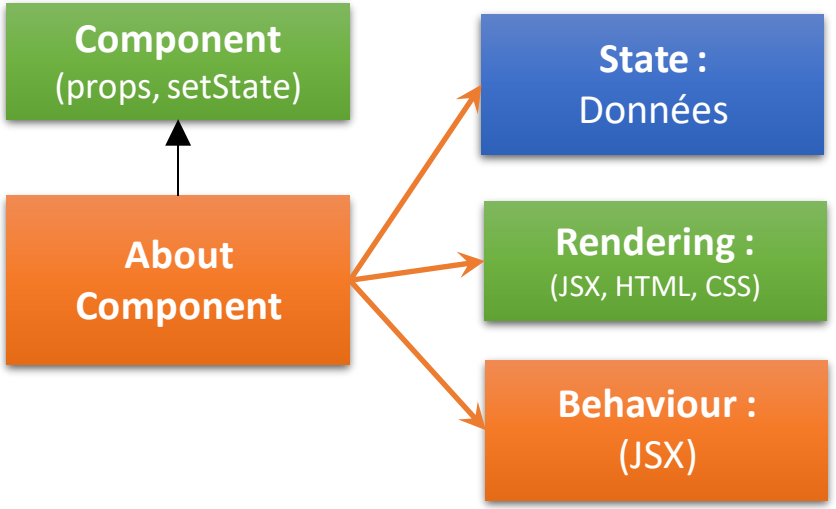
React : Composants

Les composants **React** se comportent vis-à-vis du reste de l'application comme une boîte noire, avec une API clairement définie. Ils contiennent en eux tout le nécessaire à leur bon fonctionnement : la structure, les styles et le comportement qui sont connectés par une syntaxe concise (baptisée JSX), ce qui facilite considérablement le développement.

```
import React from "react";

const styles = {
  container: { width: 500, height: 200, backgroundColor: "#0CC"
  img: { width: 50, height: 50 },
  h2: { color: "#FDE" }
};

const Demo = () => {
  return (
    <div style={styles.container}>
      <img style={styles.img} src="./logo.png" />
      <h2 style={styles.img}> {this.props.name} </h2>
    </div>
  );
};

export default Demo;
```

React : Composants

- Un composant React peut être représenté par :
  - Une fonction Java Script : **Stateless Component**
  - Une classe Héritant de la classe React.Component : **Stateful Component**
- Un composant React est définit par :
  - **Son état** (State et Props)
  - **Son Rendu** (JSX, HTML, CSS)
  - **Son Comportement** (JSX)



Composants : deux déclarations

Classe : Stateful Component

Classe héritant de `React.Component`

```
class MyComponent extends React.Component{
  // seule méthode obligatoire
  render() {
    return <h1>Hello   !</h1>;
  }         {this.props.name}

}
```

Ancienne approche, orientée objet

Considérée comme legacy (obsolète)

Fonction : Stateless Component

Fonction qui retourne du JSX

```
function MyComponent(props) {
  return <h1>Hello {props.name}!</h1>;
}
```

```
const MyComponent = ({ name }) => {
   return <h1>Hello {name} !</h1>;
}
```

Approche moderne

- Plus proche du modèle conceptuel
- Syntaxe plus légère
- Contrôle plus fin des conditions de rendu

Composants et props

Création de Composants Fonctionnels et de Classe

Composants Fonctionnels: Simples fonctions JavaScript qui acceptent des props (propriétés) et retournent du JSX

```
function MyComponent(props) {
  return <h1>Hello {props.name}!</h1>;
}
```

Composants de Classe : Classes ES6 qui étendent React.Component et implémentent une méthode render

```
class MyComponent extends React.Component{
  // seule méthode obligatoire
  render() {
    return <h1>Hello   !</h1>;
  }        {this.props.name}

}
```

Configuration des composants : Props

Composants et props

Passer des Données avec les Props

- **Qu’est-ce que les Props ?**

  Props (propriétés) sont des arguments passés aux composants React.

  Permettent de transmettre des données d'un parent à un enfant.

- **Utilisation des Props**

Exemple de Composant Parent

```
function App() {
  return <MyComponent name="lachhab"/>;
}
```

Exemple de Composant enfant

```
function MyComponent(props) {
  return <h1>Hello {props.name}!</h1>;
}
```

Composants et props

Rendu Conditionnel et Listes

- **Rendu Conditionnel :** Utilisation de conditions pour afficher différents éléments.

```
function Salutation(props) {
const estConnecte = props.estConnecte;
   if (estConnecte) {
      return <h1> Bienvenue de retour !</h1>;
}
  return <h1>Veuillez vous inscrire.</h1>;
}
```

Le but de ce composant est de fournir un message d'accueil personnalisé en fonction de l'état de connexion de l'utilisateur. C'est un exemple classique de rendu conditionnel dans React.

Composants et props

Rendu Conditionnel et Listes

- **Rendu de Listes** : Utilisation de map() pour rendre des listes d'éléments..

```
function ListeDeNombres(props) {
const nombres = props.nombres;
const itemsDeListe = nombres.map((nombre) =>
    <li key={nombre.toString()}>{nombre}</li>
);

return <ul>{itemsDeListe}</ul>;}
```

Le code présente un composant React qui prend en entrée un tableau de nombres via ses propriétés (props). Le composant utilise la méthode map pour transformer ce tableau de nombres en une liste d'éléments de liste.

Composants et props

Les props sont en lecture seule

- Que vous déclariez un composant sous forme de fonction ou de classe, il ne doit jamais modifier ses propres props.
- Cela signifie que les composants enfants ne peuvent pas modifier les props qu'ils reçoivent. Les props doivent être traitées comme **immuables**.

```
function UserProfile(props){
return (
<div> <h2>Nom : {props.name}</h2>
<p>Âge : {props.age}</p></div>
);
}

function App() {
return (
<div>
<UserProfile name="Alice" age={30}/>
<UserProfile name="Bob" age={25} />
</div>
);}
```

```
function UserProfile(props){
props.name = "John";
return (
<div> <h2>Nom : {props.name}</h2>
<p>Âge : {props.age}</p></div>
);
}
function App() {
return (
<div>
<UserProfile name="Alice" age={30}/>
<UserProfile name="Bob" age={25} />
</div>
);}
```

Composants et props

Exercices:

- **Exercice 1 :** Créez un composant fonctionnel en React qui prend un message d'avertissement en tant que prop et affiche ce message dans une boîte d'avertissement stylisée.

Attention : Vous avez des mises à jour en attente !

Erreur : Impossible de charger les données.

Composants et props

- **Exercice 2 :**
  Créer une application React qui affiche un tableau de statut pour un utilisateur en fonction de son rôle et de son niveau d'accès, avec des boutons pour changer ces valeurs. Gérer l'état de l'utilisateur et afficher différents messages.
  - Si le role est "admin" et le niveauAcces est 5 : Afficher "Vous avez un accès complet en tant qu'administrateur"
  - Si le role est modérateur et le niveauAcces est entre 3 et 4 :Afficher "Vous avez des privilèges de modérateur"
  - Si le role est "utilisateur" et le niveauAcces est 1 ou 2, affichez "Vous avez un accès limité en tant qu'utilisateur."
  - Sinon, affichez "Accès non autorisé."

Composants et props

- **Exercice 3 :**

Créer un composant qui rend une liste de tâches à partir d'un tableau passé en prop.

Configuration des composants : State

States

Les **states** (états) dans React sont des objets utilisés pour gérer les **données dynamiques** d'un composant et permettent de mettre à jour l'interface utilisateur en réponse aux changements de ces données. Contrairement aux **props**, qui sont passées par les composants parents, les **states** sont gérés **localement** par le composant et peuvent être modifiés **directement**.

States

- **C’est quoi le « state » en React ?**
  - un **objet**, où l'on va pouvoir **stocker les donnés** relatives à un composant donné et qui peuvent changer au fil du temps.
  - accessible uniquement à travers son composant.
  - utilisable uniquement dans les composants de classe “**stateful**”.
  - lorsqu'un state change, React déclenche un nouveau **rendu** (**render**) du composant, mettant ainsi à jour l'interface utilisateur en fonction du nouveau state.

States

- **Création du state :**
  - **this.state** = { définition *des données sous forme de (key : value)* };

```
this.state = {
    nom: 'Alice',                    // Chaîne de caractères (string)
    age: 25,                         // Nombre (number)
    estConnecte: true,               // Booléen (boolean)
    hobbies: ['Lecture', 'Voyage', 'Sport'],  // Tableau (array)
    adresse: {                       // Objet (object)
      rue: '123 rue Principale',
      ville: 'Paris',
      codePostal: '75001'
    }
  };
```

States

- **Utilisation du « state » :**
  - L'accès aux données stockées dans le state se fait à travers : ***this.state.nomAttribut***
  - Pour l'affichage, on utilise la notation { } (on parle d'interpolation)

```
render() {
  return (
    <div>
      <h1>Profil de l'utilisateur</h1>
      <p>Nom: {this.state.nom}</p>
      <p>Âge: {this.state.age}</p>
      <p>Statut de connexion: {this.state.estConnecte ? 'Connecté' : 'Déconnecté'}</p>
      <p>Hobbies: {this.state.hobbies.join(', ')}</p>
      <p>Adresse: {this.state.adresse.rue}, {this.state.adresse.ville}, {this.state.adresse.codePostal}</p>
    </div>
  );
}
```

Utilisation des States dans une Classe

- **Modification du « state » :**
  - Se fait via la méthode ***this.setState à*** laquelle on passera un objet avec l'attribut à modifier et la nouvelle valeur à appliquer.

```
this.state={
    monTexte: ''
}
```

```
this.setState(
    {
        monTexte:"Bonjour à toutes et à tous !"
    });
```

Utilisation des States dans une Classe

```
class CounterStateClass extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 }; // Initialisation du state
  }
```

- **this.state** est utilisé pour accéder à l'état actuel.
- **this.setState()** est utilisé pour mettre à jour l'état. Il déclenche un nouveau rendu du composant.

```
incrementCount = () => {
  this.setState({ count: this.state.count + 1 }); // Mise à jour du state
};
```

Utilisation des States dans une Classe

- **Modification du « state » :**
  - Définition de l'action au click :
    - Pour cet exemple : on met à jour l'état en incrémentant la valeur courante du compteur
    - React effectue un rendu intégral d'un composant à chaque changement des données qui lui sont associées,

```
incrementCount = () => {
  this.setState({ count: this.state.count + 1 }); // Mise à jour du state
};
```

```
render() {
  return (
    <div>
      <h1>Compteur : {this.state.count}</h1>
      <button onClick={this.incrementCount}>Incrémenter</button>
    </div>
  );
}
```

Utilisation des States dans une Fonction (avec useState)

Avec les composants fonctionnels dans React, on utilise le **Hook useState** pour gérer les états. Ce **Hook** simplifie la gestion des états dans les fonctions.

```
import React, { useState } from 'react';

function CounterStateFonc() {
  const [count, setCount] = useState(0); // Déclaration de l'état avec useState
```

- **useState** est un hook qui permet d'ajouter **l'état local** aux composants fonctionnels.
- **useState** renvoie un tableau avec deux éléments : la valeur de l'état actuel et une fonction pour mettre à jour cet état.

States

```react_class_component.md
import React, { Component } from 'react';
import './Counter.css';

class CounterStateClass extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 }; // Initialisation du state
  }

  incrementCount = () => {
    this.setState({ count: this.state.count + 1 }); // Mise à jour
  };

  render() {
    return (
      <div>
        <h1>Compteur : {this.state.count}</h1>
        <button onClick={this.incrementCount}>Incrémenter</button>
      </div>
    );
  }
}

export default CounterStateClass;
```

```react_hooks.md
import React, { useState } from 'react';
import './Counter.css';

function CounterStateFonc() {
  const [count, setCount] = useState(0); // Déclaration de l

  const incrementCount = () => {
    setCount(count + 1); // Mise à jour de l'état
  };

  return (
    <div>
      <h1>Compteur : {count}</h1>
      <button onClick={incrementCount}>Incrémenter</button>
    </div>
  );
}

export default CounterStateFonc;
```

States

Exercice : Composant de bouton bascule

- Créez un composant de classe React appelé **BoutonBascule** qui affiche un bouton avec l'étiquette ALLUMÉ ou ÉTEINT. Le bouton doit changer son état à chaque clic, c'est-à-dire que si l'état actuel est ALLUMÉ, il doit passer à ÉTEINT et vice-versa.
- Utilisez l'état local pour suivre si le bouton est allumé ou éteint.
- Ajoutez un style CSS de base pour améliorer l'apparence du bouton.